[Lab/Group 1] Reinforcement Learning

Jae Yun JUN KIM*

Lab due: Before the end of today lab session
Evaluation: By showing your code and results, and explaining them to the Professor.
Remark:

- Only teams of two or three people accepted (preferably three).
- No late evaluation is accepted.
- No plagiarism. If plagiarism happens, both the "lender" and the "borrower" will have a zero.
- Code yourself from scratch **strictly following the theory given during lecture**.
- No lab work will be considered if any ML library is used.
- Do thoroughly all the demanded tasks.
- Study the theory for the questions.

---

1 Continuous environment: Pendulum

Source:

```
https://gymnasium.farama.org/environments/classic_control/pendulum/
```

1.1 Description

Figure 1: Pendulum

The inverted pendulum swingup problem is based on the classic problem in control theory. The system consists of a pendulum attached at one end to a fixed point, and the other end being free. The pendulum starts in a random position and the goal is to apply torque on the free end to swing it into an upright position, with its center of gravity right above the fixed point.

The diagram below specifies the coordinate system used for the implementation of the pendulum's dynamic equations.

*ECE Paris, School of Engineering, 10 Rue Sextius Michel 75015 Paris, France; jae-yun.jun-kim@ece.fr

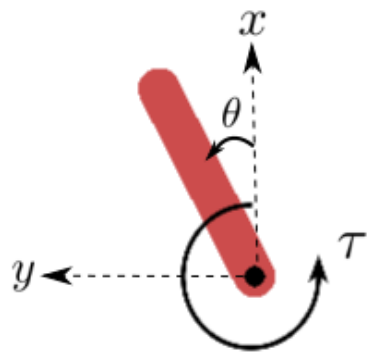

Figure 2: Pendulum (explained)

- `x-y`: cartesian coordinates of the pendulum's end in meters.
- θ: angle in radians.
- τ: torque in N m. Defined as positive counter-clockwise.

1.1.1 Action space

The action is a ndarray with shape (1,) representing the torque applied to free end of the pendulum.

Num	Action	Min	Max
0	Torque	-2.0	2.0

Figure 3: Action space

1.1.2 Observation space

The observation is a ndarray with shape (3,) representing the x-y coordinates of the pendulum's free end and its angular velocity.

Num	Observation	Min	Max
0	x = cos(theta)	-1.0	1.0
1	y = sin(theta)	-1.0	1.0
2	Angular Velocity	-8.0	8.0

Figure 4: Observation space of pendulum

1.1.3 Rewards

The reward function is defined as:

$$r = -(\text{theta}^2 + 0.1 * \text{theta_vel}^2 + 0.001 * \text{torque}^2)$$

where theta is the pendulum's angle normalized between [-pi, pi] (with 0 being in the upright position). Based on the above equation, the minimum reward that can be obtained is $-(pi^2 + 0.1 * 8^2 + 0.001 * 2^2) = -16.2736044$, while the maximum reward is zero (pendulum is upright with zero velocity and no torque applied).

1.1.4 Starting state

The starting state is a random angle in [-pi, pi] and a random angular velocity in [-1,1].

1.1.5 Episode end

The episode truncates at 200 time steps.

1.1.6 Arguments

Pendulum has two parameters for gymnasium.make with `render_mode` and g representing the acceleration of gravity measured in (m s-2) used to calculate the pendulum dynamics. The default value is g = 10.0. On reset, the options parameter allows the user to change the bounds used to determine the new random state.

```
import gymnasium as gym
env = gym.make("Pendulum-v1", render_mode="rgb_array", g=9.81)  # default g=10.0
print(env)
```

1.2 Lab tasks

1. Following strictly the theory given in class, using `Q-learning` algorithm, for all states and actions, find the optimal action-value function table (i.e., Q^*-table) taking into account the following two notes:

   a. You need to discretize the state values since they are originally real numbers. You can do so by using functions such as `np.linspace()`, and `np.digitize()`.

   b. Store the optimal Q-table in a file using

   ```
   import pickle
   f = open("mountain_car.pkl","wb")
   pickle.dump(q,f) # where q refers to the optimal Q-table
   f.close()
   ```

   If you prefer other methods to store the model parameters in a file, you can choose those methods as well.

   c. To efficiently train your model, avoid the animation by giving the value `None` to `render_mode`.

2. Test the optimality of your model by loading the stored optimal Q-table and by showing the associated animation results (i.e., the `render_mode` should have `human` as its value for this purpose). To load the stored optimal Q-table, you can do so by typing

   ```
   f = open('mountain_car.pkl', 'rb')
   q = pickle.load(f)
   f.close()
   ```

   If you prefer other methods to store the model parameters in a file, you can choose those methods as well.

2 Discrete environment: Taxi

Source:

```
https://gymnasium.farama.org/environments/toy_text/taxi/
```

2.1 Description

This environment is part of the Toy Text environments which contains general information about the environment. and is shown in Figure 5.

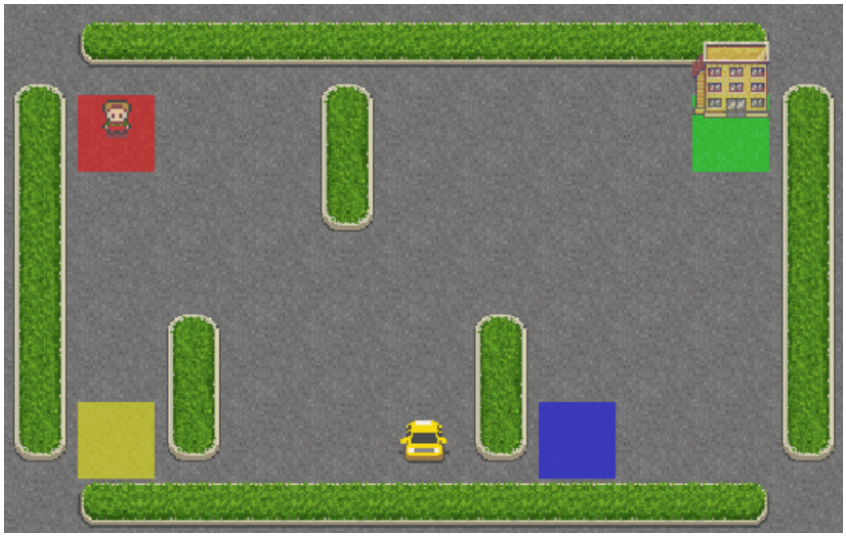

Figure 5: Taxi

The Taxi Problem involves navigating to passengers in a grid world, picking them up and dropping them off at one of four locations.

There are four designated pick-up and drop-off locations (Red, Green, Yellow and Blue) in the 5x5 grid world. The taxi starts off at a random square and the passenger at one of the designated locations. The goal is move the taxi to the passenger's location, pick up the passenger, move to the passenger's desired destination, and drop off the passenger. Once the passenger is dropped off, the episode ends. The player receives positive rewards for successfully dropping-off the passenger at the correct location. Negative rewards for incorrect attempts to pick-up/drop-off passenger and for each step where another reward is not received.

2.1.1 Action space

The action shape is (1,) in the range 0, 5 indicating which direction to move the taxi or to pickup/drop off passengers:

- 0: Move south
- 1: Move north
- 2: Move east
- 3: Move west
- 4: Pickup passenger
- 5: Drop off passenger

2.1.2 Observation space

There are 500 discrete states since there are 25 taxi positions, 5 possible locations of the passenger (including the case when the passenger is in the taxi), and 4 destination locations.

Destination on the map are represented with the first letter of the color.

Passenger locations:

- 0: Red
- 1: Green
- 2: Yellow
- 3: Blue
- 4: In taxi

Destinations:

- 0: Red
- 1: Green
- 2: Yellow
- 3: Blue

An observation is returned as an int() that encodes the corresponding state, calculated by:

$$((\text{taxi_row} * 5 + \text{taxi_col}) * 5 + \text{passenger_location}) * 4 + \text{destination}$$

Note that there are 400 states that can actually be reached during an episode. The missing states correspond to situations in which the passenger is at the same location as their destination, as this typically signals the end of an episode. Four additional states can be observed right after a successful episodes, when both the passenger and the taxi are at the destination. This gives a total of 404 reachable discrete states.

2.1.3 Starting state

The initial state is sampled uniformly from the possible states where the passenger is neither at their destination nor inside the taxi. There are 300 possible initial states: 25 taxi positions, 4 passenger locations (excluding inside the taxi) and 3 destinations (excluding the passenger's current location).

2.1.4 Rewards

- -1 per step unless other reward is triggered.
- +20 delivering passenger
- -10 executing "pickup" and "drop-off" actions illegally.

An action that results a noop, like moving into a wall, will incur the time step penalty.

2.1.5 Episode end

The episode ends if the following happens:

- Termination: 1. The taxi drops off the passenger.
- Truncation (when using the time_limit wrapper): 1. The length of the episode is 200.

2.1.6 Arguments

```
import gymnasium as gym
gym.make('Taxi-v3')
```

2.2 Lab tasks

1. Following strictly the theory given in class, using `Q-learning` algorithm, for all states and actions, find the optimal action-value function table (i.e., Q^* table) taking into account the following two notes:

   a. Store the optimal Q-table in a file using

   ```
   import pickle
   f = open("taxi.pkl","wb")
   pickle.dump(q,f) # where q refers to the optimal Q-table
   f.close()
   ```

   If you prefer other methods to store the model parameters in a file, you can choose those methods as well.

   b. To efficiently train your model, avoid the animation by giving the value `None` to `render_mode`.

2. Test the optimality of your model by loading the stored optimal Q-table and by showing the associated animation results (i.e., the `render_mode` should have `human` as its value for this purpose). To load the stored optimal Q-table, you can do so by typing

   ```
   f = open('taxi.pkl', 'rb')
   q = pickle.load(f)
   f.close()
   ```

   If you prefer other methods to store the model parameters in a file, you can choose those methods as well.